

CO, AT, 1

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1714

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I SRINIVASAN . A / 192411025 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A. Srinivasan

88
18

Self Driving car on city Roads:

A Self-driving car is an intelligent agent that observes its surroundings and takes suitable actions to reach its destination safely.

Percepts:

- * Camera - detects vehicles, pedestrians and traffic signals.
- * LIDAR/Radar - measures distance and detects obstacles.
- * GPS - Provides location and route information.
- * Sensors - detects speed, position and road conditions.

Possible Actions:

- * Accelerate and slow down
- * Apply brakes and stop.
- * Steer left or right
- * change lanes and turn
- * Overtake when safe
- * Select the best route.

Environment:

- * Partially observable: Some objects may be hidden
- * Stochastic: other drivers and pedestrians behave unpredictably.

* Dynamic : Traffic and road conditions continuously change.

* continuous : speed, position and steering are continuous.

* Multi-agent : other vehicles, cyclists and pedestrians are present.

Suitable Agent: Utility-Based Agent;

A utility-based agent is most suitable because it selects actions by considering safety, travel time, comfort and fuel efficiency.

Conclusion : The self-driving car works in a partially observable, stochastic, dynamic and continuous environment. A utility based agent provides safer and more efficient decisions.

② Cleaning Robot - online Search Agent:

An online search agent is useful when a cleaning robot enters an unknown building without a map. working:

1. Robot starts from an initial location.
2. It senses nearby walls, doors and obstacles.
3. It selects an unexplored path.
4. It moves and observe the result.
5. It records the discovered information.

6. It updates its internal map
- 7. It avoids unnecessary repeated path
 - 8. It continues until the required area is explored and cleaned.

challenges:

- * No prior map is available
- * The robot can sense only nearby areas
- * It may encounter unknown obstacles
- * Some paths may lead to dead ends
- * It may revisit already explored locations
- * Battery power and time are limited.

Online vs offline Search

Online search

Environment is initially
Unknown

learns while moving

Uses sensors continuously

May make inefficient
moves

Offline Search

Environment is known

Plans before moving

Uses available information

Can find a path in advance

Conclusion: Online Search is suitable because the robot must sense, act, learn and explore at the same time.

③ Bidirectional Search vs BFS

Breadth-First Search (BFS) starts at S and searches level by level until it reaches G .

Bidirectional Search searches simultaneously from both ends:

- * Forward from S
- * Backward from G
- * Stops when both searches meet.

Complexity

For BFS:

$$O(b^d)$$

For Bidirectional Search:

$$O(b^{d/2})$$

where b is the branching factor and d is the depth of the solution.

Example:

consider the path:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$

BFS:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$

It searches from only one direction.

Bidirectional Search :

$S \rightarrow A \rightarrow B \leftarrow C \leftarrow D \leftarrow G$

The two searches meet near the middle, so fewer nodes are explored.

Advantages of Bidirectional Search.

- * Faster for deep search problems
- * Explores fewer nodes
- * Reduces search time
- * can find the shortest path when conditions

Conclusion: Bidirectional Search is ^{are suitable.} generally more efficient than BFS for deep graphs because it searches from both the start and goal, reducing the effective search depth approximately by half.